8086 Unconditional Jump Instructions

1. Introduction to JMP

Concept

JMP (Jump) transfers program control unconditionally to another instruction address. It allows the programmer to:

- skip sections of code
- create loops
- branch execution to another part of memory

Rules / Notes

Types of Unconditional Jumps in 8086

- **Short Jump**
- **Near Jump**
- **Far Jump**

Classification:

- Short & Near → **Intra-segment jumps**
- Far → **Inter-segment jump**

2. Short Jump

Concept

A **Short Jump** is a **relative jump** within the current code segment. It can jump forward or backward within a limited range.

Rules / Notes

Characteristics

- Jump range: **+127 to −128 bytes**
- Displacement is **signed 8-bit**
- Jump address is **not stored explicitly**
- Target address is computed as:

$$\text{Target Address} = IP + (\text{sign-extended displacement})$$

Example

```
XOR BX, BX
START: MOV AX, 1
       ADD AX, BX
       JMP SHORT NEXT
       ; skipped instructions
NEXT:  MOV BX, AX
       JMP SHORT START
```

Explanation:

- `JMP SHORT NEXT` skips intermediate code
- Displacement is calculated relative to current IP
- Execution remains within the same code segment

Exam Tip

Short jumps are compact (2 bytes) and faster. Used when target is close.

3. Near Jump

Concept

A **Near Jump** transfers control to any location within the **current code segment**.

Rules / Notes

Characteristics

- Jump range: **±32 KB**
- Instruction size: **3 bytes**
- Uses **signed 16-bit displacement**
- Address calculation:

$$\text{Target Address} = IP + (\text{signed 16-bit displacement})$$

Key Property: Near jumps are **relative and relocatable**. If the code segment is moved, the jump still works.

Example

```
XOR BX, BX
START: MOV AX, 1
       ADD AX, BX
       JMP NEXT
       ; skipped instructions
NEXT:  MOV BX, AX
       JMP START
```

Explanation:

- Jump remains within the same code segment
- Distance between source and target is preserved

Exam Tip

Near jumps are preferred when jump distance is large but still within the same code segment.

4. Far Jump

Concept

A **Far Jump** transfers execution to a different code segment. It is an **inter-segment jump**.

Rules / Notes

Characteristics

- New **CS** and **IP** are loaded
- Instruction contains:
  - Offset (bytes 2–3)
  - Segment (bytes 4–5)
- Uses **FAR PTR** or far label

Example

```
EXTRN UP:FAR

XOR BX, BX
START: ADD AX, 1
       JMP NEXT
       ; skipped instructions
NEXT:  MOV BX, AX
       JMP FAR PTR START
       JMP UP
```

Explanation:

- `JMP FAR PTR START` reloads CS:IP
- `JMP UP` jumps to a far label in another segment

Exam Tip

Far jumps are used when transferring control between segments (e.g., overlay programs, modular code).

Comparison Summary

- **Short Jump:** ±128 bytes, 8-bit displacement
- **Near Jump:** ±32 KB, 16-bit displacement
- **Far Jump:** new CS:IP loaded

Exam Tip

Short and Near jumps are **intra-segment**. Far jumps are **inter-segment**.